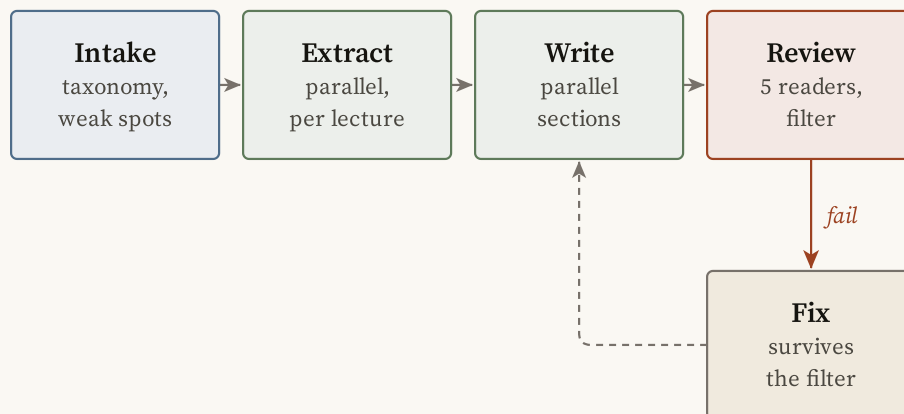


pdf-material-builder

A skill that turns course material into LaTeX study documents in one house style, checked by five reviewers before it's called done.

The output is any of the usual study documents: a reference doc, a formula sheet, worked examples, visual intuition, course notes, a companion to a lesson, a cheat sheet, a technical doc like this one. Left to separate write-ups, these drift: one document uses a different notation for the same symbol than the next, a claim about a theorem goes unchecked, and the voice shifts depending on who or what wrote the section. pdf-material-builder is one pipeline for all of them, where the only real branch point is size: a large course pack runs extraction and eight or so parallel writers, a small one skips straight to drafting.

FIGURE 1 *the parts*



Nothing gets fixed until the filter has checked it; only what survives goes back to a writer.

Extraction and writing both run in parallel; review is what stands between a draft and a pack that ships.

The parts

PIECE	WHAT IT DOES
SKILL.md	The orchestrator. Decides large or small from references/recipes.md and runs the matching pipeline.
Extractors	Around eight parallel agents, one per lecture, writing per-lecture markdown into _extraction/.
Conventions doc	The single source of truth for notation, built once from the extraction and cited first in every writer's brief.
Anchor reference doc	Written first, so later documents have something settled to point back to.
Section writers	Eight to ten parallel agents per document, working from a driver that pre-declares each Part so two writers never claim the same number.
Five reviewers	Full-context content, a brutal student peer, cross-doc consistency, math verification in SymPy, and a cold-edit reader with minimal context.
The filter	Drops what the reviewers found that doesn't hold up, by re-deriving math, grepping quotes, and requiring two reviewers to agree on a subjective claim.
voice-drift.sh	Fails the build if a section's voice has drifted from the house style vendored from lesson-builder.

How one pack is built

1. **Intake.** A short set of questions fixes the taxonomy, the learner's weak areas, past exams, and any topic that's off limits. This is what tells the rest of the pipeline what to build and what to skip.
2. **Extract.** For a large build, roughly eight extractors run in parallel, one per lecture, each writing what it found to `_extraction/` rather than straight into a document.
3. **Set conventions.** One conventions doc is written from the extraction and becomes the single source of truth; every writer's brief cites it first, before anything else.
4. **Anchor.** The reference doc is written before the rest, so the documents that come after have a settled thing to point back to instead of each defining its own notation.
5. **Draft.** The orchestrator writes a driver that pre-declares every Part, then spawns eight to ten section writers in parallel against it. A small build skips extraction and drafts straight from the source material.
6. **Review.** Five reviewers read the draft in parallel: full-context content, a brutal student peer, cross-doc consistency against a canonical-equations table grepped across every `.tex` file, math verification in SymPy reporting pass, fail, or unverifiable, and a cold-edit reader given minimal context.
7. **Filter and fix.** A mandatory pass drops the findings that don't hold up before anyone fixes anything, then `scripts/build.sh` runs three `lualatex` passes and the style and page-break checks. A small build runs the same filter with two reviewers instead of five.

The hard parts

3.1 A filter between the reviewers and the fixes

Five reviewers reading in parallel find a lot that isn't real: a false claim about notation that turns out to match the conventions doc, or two reviewers independently unhappy about the same passage for different, unrelated reasons. The filter re-derives any math claim by hand instead of trusting the reviewer's word, greps a quoted string against the actual file before accepting that it's wrong, and requires two reviewers to agree before a subjective complaint counts. It also checks the blast radius of a finding, since a fix that touches the conventions doc can ripple into every other document in the pack. A finding that survives all that goes back to a writer; the rest are dropped before anyone edits a file.

3.2 One notation, chosen once, and Parts that don't collide

Parallel writers left to their own judgment pick different symbols for the same quantity. The conventions doc makes that choice once, cited first in every writer's brief, ahead of the source material; cross-doc consistency review then greps a canonical-equations table across every .tex file, so drift that slipped past the brief still gets caught. The same problem shows up in numbering: eight to ten writers each drafting a section would collide on the same Part number if left to decide independently, so the orchestrator's driver pre-declares every Part first and a writer only fills in a slot that already has a fixed number.

3.3 Math checked, not just read

A reviewer that only reads a derivation can miss a sign error, especially the kind that recurs across a course because everyone repeats the same mistake. One of the five reviewers runs the math through SymPy and reports pass, fail, or unverifiable rather than a prose opinion. Courses accumulate their own catalogue of false positives this way too. A known case is a reviewer that keeps flagging a Richardson extrapolation sign as wrong when it is the convention that's unusual, not the derivation.

What it doesn't do

- There is no path from a .docx source to a PDF; there's no LibreOffice in the pipeline to do that conversion.
- Publishing a companion document depends on changes landing in other repos first; it isn't a step this skill can finish on its own.
- CI can't fetch the private diagram-maker submodule, so the tests that depend on it skip there rather than run.
- A small build runs two reviewers, not five, and skips extraction; it trades coverage for speed on purpose.

From the repo <https://github.com/ihsan-sa/pdf-material-builder>: SKILL.md; references/recipes.md; references/review-pipeline.md; scripts/build.sh; scripts/voice-drift.sh; scripts/page-break-check.sh; scripts/style-check.sh; tests/check.sh, read 26 September 2026.